

Paper Code(s): ECC-207	L	P	C
Paper: Digital Logic and Computer Design	4	-	4

Marking Scheme:

1. Teachers Continuous Evaluation: 25 marks
2. Term end Theory Examinations: 75 marks

Instructions for paper setter:

1. There should be 9 questions in the term end examinations question paper.
2. The first (1st) question should be compulsory and cover the entire syllabus. This question should be objective, single line answers or short answer type question of total 15 marks.
3. Apart from question 1 which is compulsory, rest of the paper shall consist of 4 units as per the syllabus. Every unit shall have two questions covering the corresponding unit of the syllabus. However, the student shall be asked to attempt only one of the two questions in the unit. Individual questions may contain upto 5 sub-parts / sub-questions. Each Unit shall have a marks weightage of 15.
4. The questions are to be framed keeping in view the learning outcomes of the course / paper. The standard / level of the questions to be asked should be at the level of the prescribed textbook.
5. The requirement of (scientific) calculators / log-tables / data – tables may be specified if required.

Course Objectives :

1. To introduce basic concepts of Boolean Algebra and Combinational Logic
2. To introduce various sequential circuits, designing with examples
3. To relate combination circuit design and sequential circuit design with respect to the design of a computer system
4. To introduce machine learning, computer arithmetic, modes of data transfer with respect to I/O and Memory organization of a computer

Course Outcomes (CO) :

- CO 1 Ability to understand Boolean Algebra and Design Combinational Circuits .
- CO 2 Ability to understand and Design Sequential Circuits.
- CO 3 Ability to understand Design of a basic computer.
- CO 4 Ability to understand Input-Output and Memory Organization of a Computer.

Course Outcomes (CO) to Programme Outcomes (PO) mapping (scale 1: low, 2: Medium, 3: High)

	PO01	PO02	PO03	PO04	PO05	PO06	PO07	PO08	PO09	PO10	PO11	PO12
CO 1	3	2	3	2	2	-	-	-	3	2	2	3
CO 2	3	2	3	2	2	-	-	-	3	2	2	3
CO 3	3	2	3	3	2	-	-	-	3	2	2	3
CO 4	3	3	3	3	3	-	-	-	3	2	2	3

UNIT – I

Boolean Algebra and Combinational Logic: Review of number systems , signed, unsigned, fixed point, floating point numbers, Binary Codes, Boolean algebra – basic postulates, theorems , Simplification of Boolean function using Karnaugh map and Quine-McCluskey method – Implementations of combinational logic functions using gates, Adders, Subtractors, Magnitude comparator, encoder and decoders, multiplexers, code converters , parity generator/checker, implementation of combinational circuits using multiplexers.

UNIT – II

Sequential Circuits: General model of sequential circuits, Flip-flops, latches , level triggering, edge triggering, master slave configuration , concept of state diagram , state table, state reduction procedures , Design of synchronous sequential circuits , up/down and modulus counters , shift registers, Ring counter , Johnson counter , timing diagram , serial adder , sequence detector, Programmable Logic Array (PLA), Programmable Array Logic (PAL), Memory Unit, Random Access Memory

UNIT – III

Basic Computer organization: Stored Program, Organization, Computer registers, bus system, instruction set completeness, instruction cycle, Register Transfer Language, Arithmetic, Logic and Shift Micro-operations, Instruction Codes, Design of a simple computer, Design of Arithmetic Logic unit, shifter, Design of a simple hardwired control unit, Programming the basic computer, Machine language instructions, assembly language, Microprogrammed control, Horizontal and Vertical Microprogramming, Central Processing Unit, instruction sets and formats, addressing modes, data paths, RISC and CISC characteristics.

UNIT – IV

Computer Arithmetic, addition, subtraction, multiplication and division algorithms, Input-Output Organization, Modes of data transfer, Interrupt cycle, direct memory access, Input-Output processor, Memory Organization, Memory Hierarchy, Associative Memory, Cache Memory, Internal and external Memory, Virtual Memory.

Text Book(s)

1. M. Morris Mano, "Digital Logic and Computer Design", Pearson Education, 2016
2. M. Morris Mano, Rajib Mall "Computer System Architecture", 3rd Edition Pearson Education, 2017

References:

1. Leach, D. P., Albert P. Malvino, "Digital Principles and Applications", McGraw Hill Education, 8th Edition, 2014
2. Jain, R.P., "Modern Digital Electronics", McGraw Hill Education, 4th Edition, 2010
3. Floyd, Thomas L., "Digital Fundamentals" Pearson Education, 11th Edition, 2017
4. M. Rafiquzzaman, "Fundamentals of Digital Logic and Microcomputer Design", Wiley, 5th Ed., 2005.

Paper Code(s): ECC-253	L	P	C
Paper: Digital Logic and Computer Design Lab	-	2	1

LIST OF EXPERIMENTS

1. Design and implementation of adders and subtractors using logic gates
2. Design and implementation of 4- bit binary adder/subtractor
3. Design and implementation of multiplexer and demultiplexer
4. Design and implementation of encoder and decoder
5. Construction and verification of 4-bit ripple counter and mod-10/mod-12 ripple counter
6. Design and implementation of 3-bit synchronous up/down counter
7. Write an assembly language code in GNUSim 8085 to implement data transfer instructions
8. Write an assembly language code in GNUSim 8085 to add two 8 bit numbers
9. Write an assembly language code in GNUSim 8085 to find the factorial of a number
10. Write an assembly language code in GNUSim 8085 to implement logical instruction

Asst.

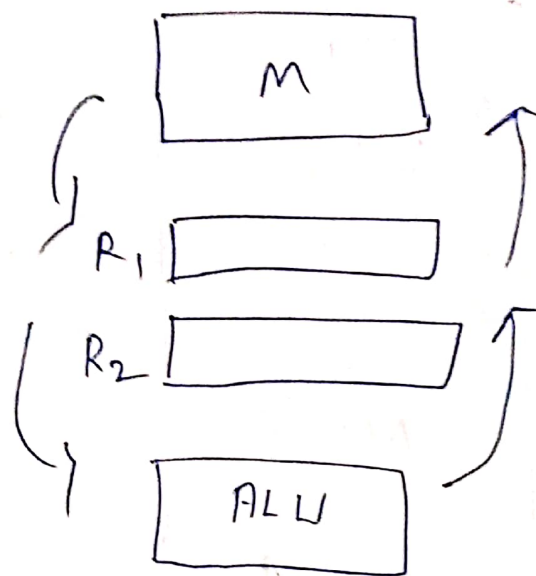
Types of computer → General purpose [Personal PC's, desktops]
 specific task → Embedded system (Washing machine, Fridge)

→ How computer is organized. (How computer is designed internally → RAM, ROM, Cache, ROM)

Memory

Registers

ALU



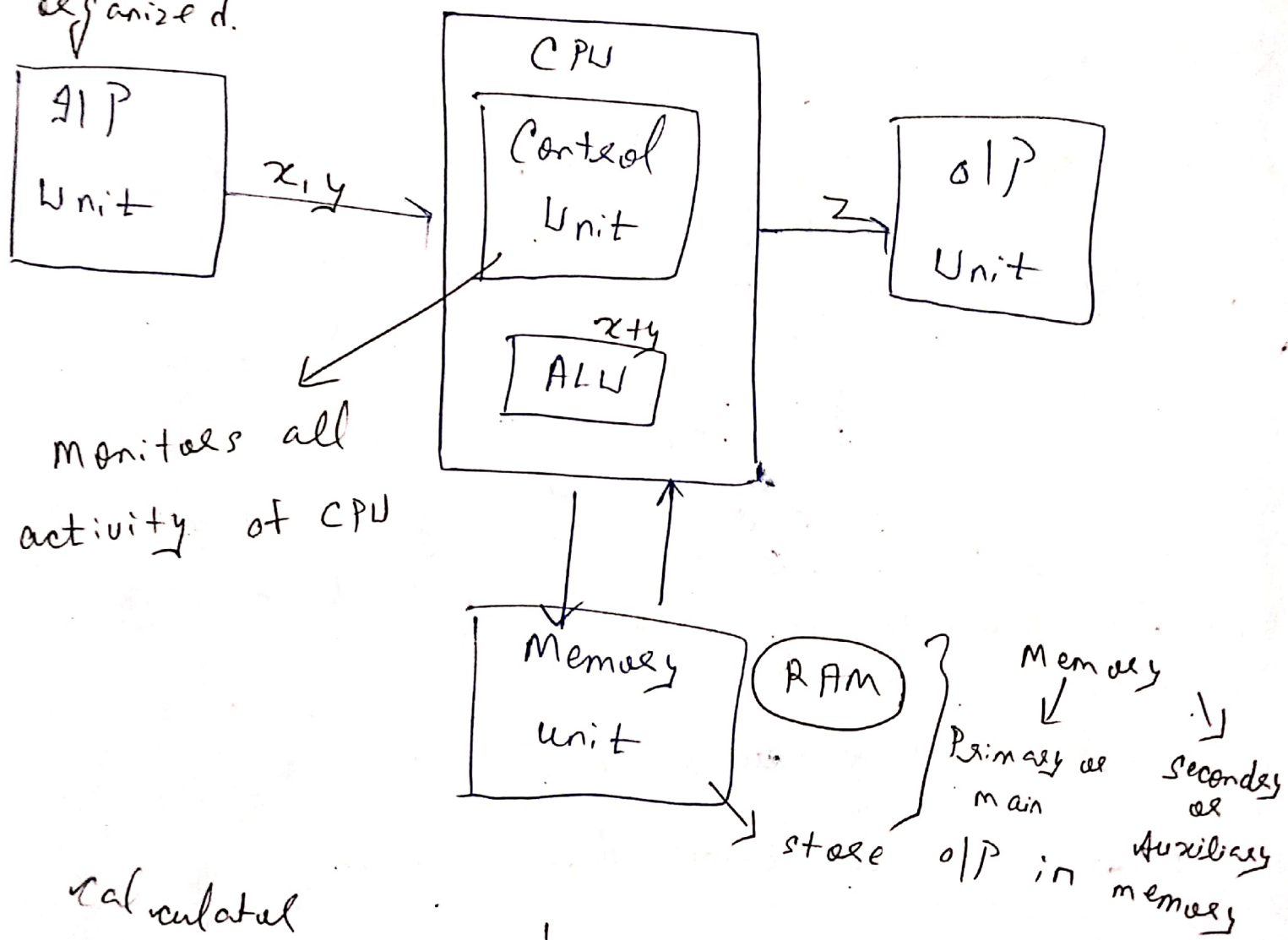
Control unit
 ↓
 at what time
 GIP o/p to be
 given & how o/p
 is to be executed.

What is computer organization:-

- ① How does the computer implement various tasks.
- ② Deals with low level design issues (logic &ckt).
- ③ structural relationship among various components.
- ④ First computer architecture is designed & then organization is done.

(2) Computer organization involves ^{at} design, machine lo
 ALU, CPU & memory.

Computer Organization
 How internal parts are organized.
 Block diagram



Calculated

$$x = 5, y = 10$$

↓

$$z = x + y = 15$$

I/P unit :- user interacts with computer.

O/P unit :- ^{used by} computer to interact with user.

(3)

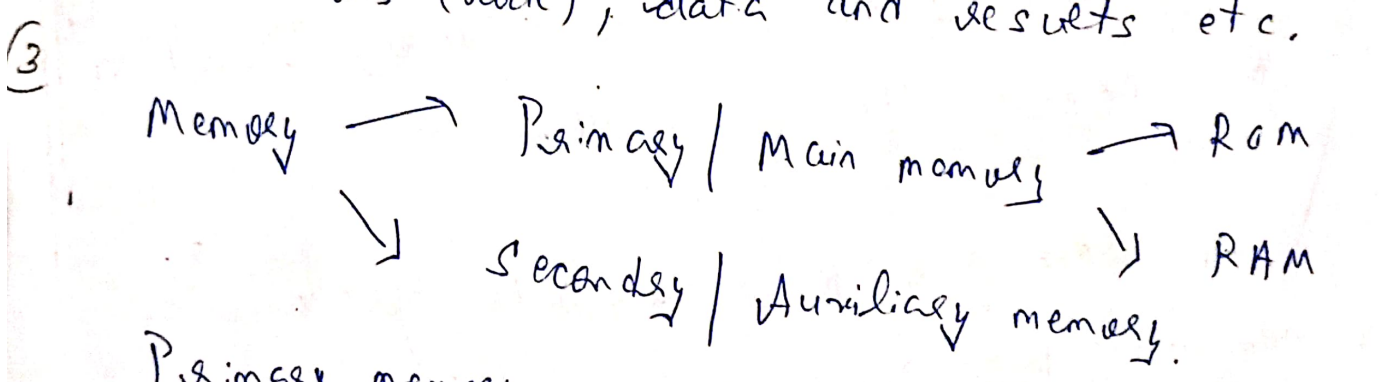
What are machine and assembly languages?

Machine language is a language that has a binary form. It can be directly executed by a computer. While an assembly language is a low level programming language that requires software called an assembler to convert it into machine code.

Machine language:- Language understood by computers.

(2) Memory unit:- This unit stores the program instructions (code), data and results etc.

UNIT-4



Primary memory :- Semiconductor memory that provides access at high speed. Run time program instructions and operands are stored in the main memory. ROM holds system programs & firmware routines that are essential to manage hardware of computer.

(4)

Memory Organization

Memory :- It is a storage unit $\rightarrow$ a place to hold data and instructions.

Memory location :- Memory is divided into multiple small parts, of fixed size & capable of holding info.



Memory address :- Each memory location is uniquely identified by a binary address.

↑
HEX

Byte addressable memory :- Each memory address refers to a single byte of storage. To store data of larger size, multiple consecutive locations are used.

Word addressable memory :-

* Each memory location can store data of size more than 1 byte.

1W = 2B

1W = 4B

* Depends on how 1 word is defined.

Types of memory :-

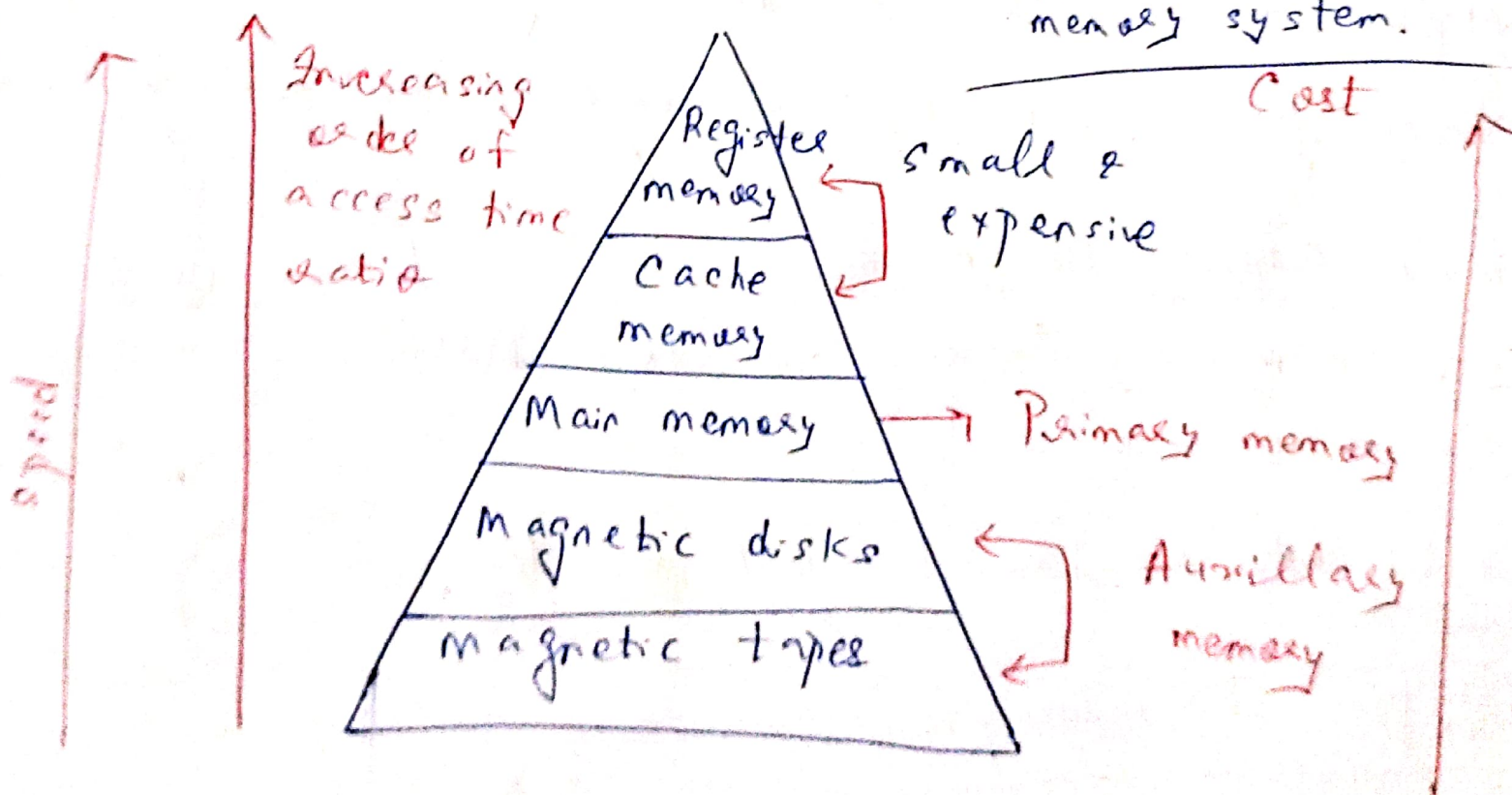
① Main or Primary memory :- Holds data & programs currently being used by C.P.U.

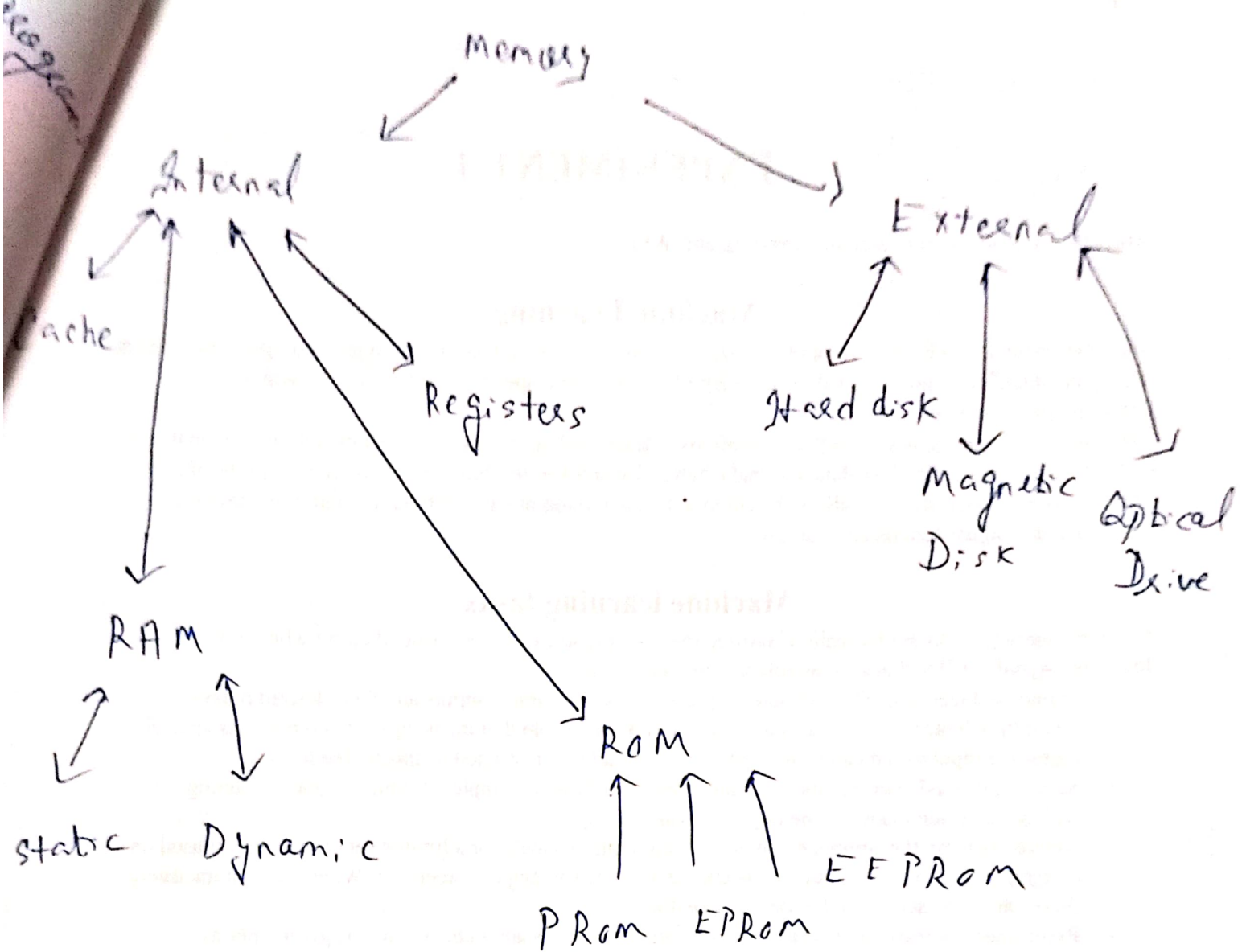
② Auxiliary or Secondary memory :- stores all those programs / data that is not required immediately by CPU.

③ Cache memory :- Used for fast retrieval of data / instructions.

Memory organization / Memory Hierarchy

To obtain highest possible access speed while minimizing the total cost of memory system.





→ Internal memory or main or primary memory

→ External or secondary memory

Associative memory / Content addressable memory (CAM) or Associative array

→ It is a special type of memory that is optimized for performing searched through data, as opposed to providing a simple direct access to the data based on the address.

EXPERIMENT-1

Objective: Introduction to machine learning and WEKA.

Machine Learning

Machine learning is a field of computer science that uses statistical techniques to give computer systems the ability to "learn" (i.e., progressively improve performance on a specific task) with data, without being explicitly programmed.

The process of learning begins with observations or data, such as examples, direct experience, or instruction, in order to look for patterns in data and make better decisions in the future based on the examples that we provide. The primary aim is to allow the computers learn automatically without human intervention or assistance and adjust actions accordingly.

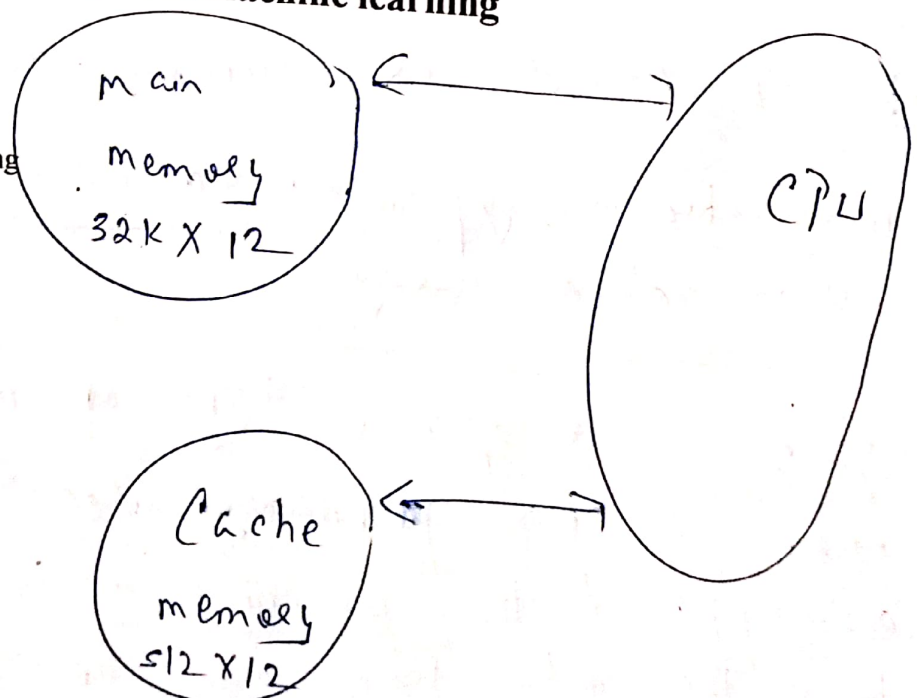
Machine learning tasks

Machine learning tasks are typically classified into two broad categories, depending on whether there is a learning "signal" or "feedback" available to a learning system:

- **Supervised learning:** The computer is presented with example inputs and their desired outputs, given by a "teacher", and the goal is to learn a general rule that maps inputs to outputs. As special cases, the input signal can be only partially available, or restricted to special feedback:
- **Semi-supervised learning:** the computer is given only an incomplete training signal: a training set with some (often many) of the target outputs missing.
- **Active learning:** the computer can only obtain training labels for a limited set of instances (based on a budget), and also has to optimize its choice of objects to acquire labels for. When used interactively, these can be presented to the user for labelling.
- **Reinforcement learning:** training data (in form of rewards and punishments) is given only as feedback to the program's actions in a dynamic environment, such as driving a vehicle or playing a game against an opponent.
- **Unsupervised learning:** No labels are given to the learning algorithm, leaving it on its own to find structure in its input. Unsupervised learning can be a goal in itself (discovering hidden patterns in data) or a means towards an end (feature learning).

Applications of machine learning

1. Virtual Personal Assistants
2. Traffic Predictions
3. Videos Surveillance
4. Face Recognition
5. Email Spam and Malware Filtering
6. Search Engine Result Refining
7. Product Recommendations
8. Online Fraud Detection
9. Image classification
10. Medical Diagnosis



Virtual memory :- It is a common technique

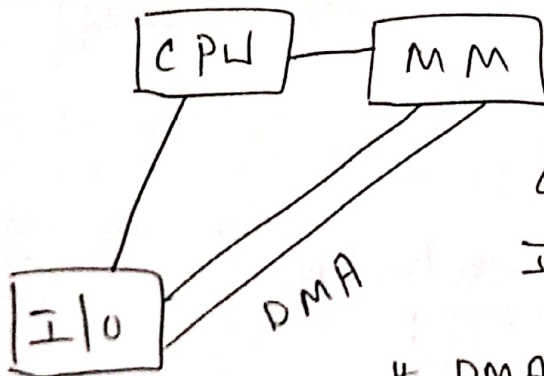
used in a computer's operating system (OS).

Virtual memory uses both $11/w$ & slw to enable a computer to compensate for physical memory shortages, temporarily transferring data from RAM to disk storage.

Features of virtual memory :-

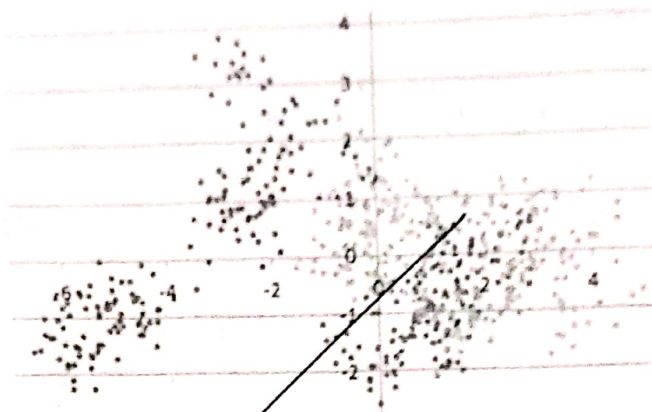
- ① They can increase memory as multiple programs & apps can run or execute simultaneously.
- ② Virtual memory can store programs that currently are not used by physical memory.
- ③ Space of virtual memory can be extended using OS features.

DMA (Direct Memory Access)



& In modern computers, DMA is used to transfer large blocks of data at high speed b/w the I/O devices & main memory.

& DMA $\uparrow$ data transfer rate

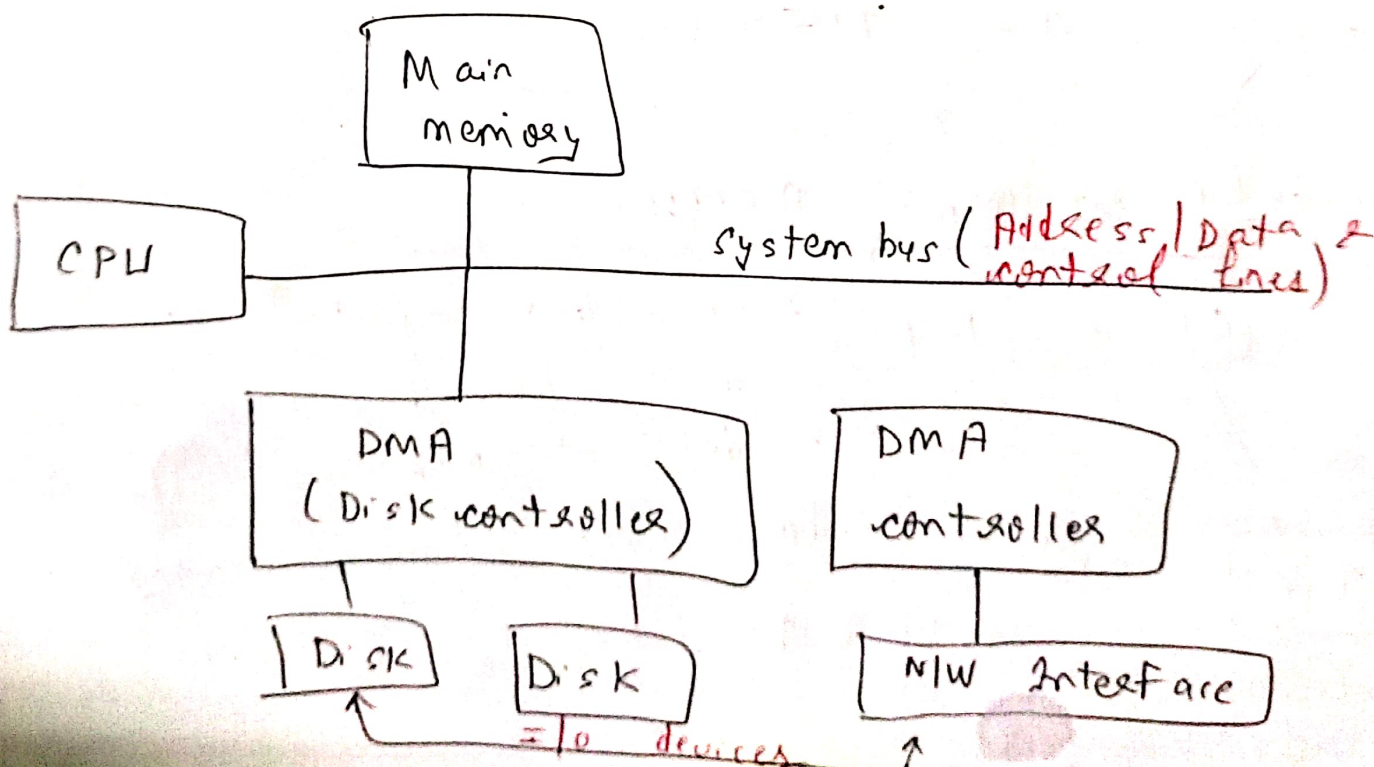


Simulated cluster structure to test the elbow rule

Stopping rule for clustering algorithms

One open question is how the methodology performs when the data has more than two dimensions. The issue is not with the elbow curve itself, but with the criterion being used. Finally, when large clusters are found in a data set (especially with hierarchical clustering algorithms) it is a good idea to apply the elbow rule to any big cluster (split the big cluster into smaller clusters), in addition to the whole data set. In practice, once you hit the first red bar (or if there is another red bar just after the first one, and bigger than the first one), you can stop refining and splitting your clusters: you have reached an empirical optimum.

* DMA allows I/O devices to transfer data directly to or from main memory without CPU's intervention. (or simply by-passing the CPU from path).



* DMA is a I/O technique that provides direct access to the main memory while CPU is temporarily disabled, to speed up the memory operations.

This process is managed by a chip known as DMA controller (DMAC).

* I/O devices are connected to system bus via a special interface circuit called "DMA controller".

* In DMA, both CPU & DMA controller have access to main memory via a shared system bus having data, address & control lines.

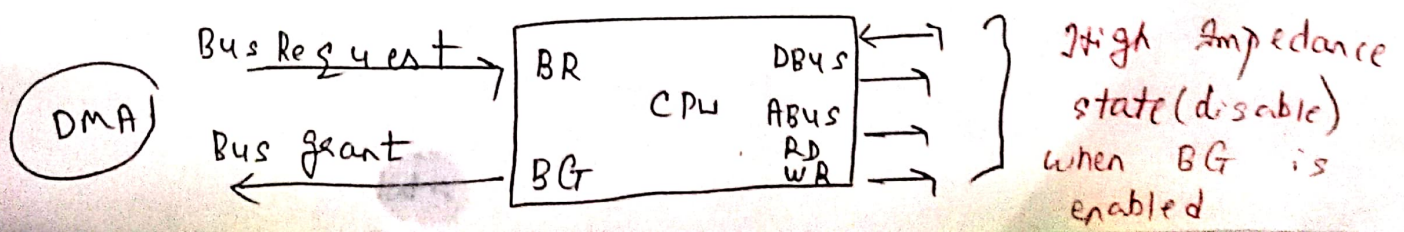
* During DMA transfer the CPU is idle & has no control of the system bus.

* "DMA controller" temporarily borrows the address bus, data & control bus from the CPU & transfers data b/w I/O devices & main memory.

* DMA transfer is used for high speed memory to memory transfer. eg USB drive.

* DMA or DMA channel.

How to make CPU idle ; -



A working

- I/O wants to transfer with MM.
- I/O sends DMA request to DMA controller (DMAC).
- DMAC sends BR (Bus request) signal to request CPU for the buses.
- DMAC waits until CPU sends BG (Bus grant) signal to DMA.
- CPU relinquishes control of buses & places address bus, data bus, RD & WR lines into a high impedance state.
- CPU activates the BG signal & becomes idle.
- DMAC takes control of the buses to conduct direct memory transfer without CPU intervention.
- When DMA transfer terminates, it disables BR.
- CPU disables BG, takes control of the buses, & returns to its normal operation.

Types of DMA

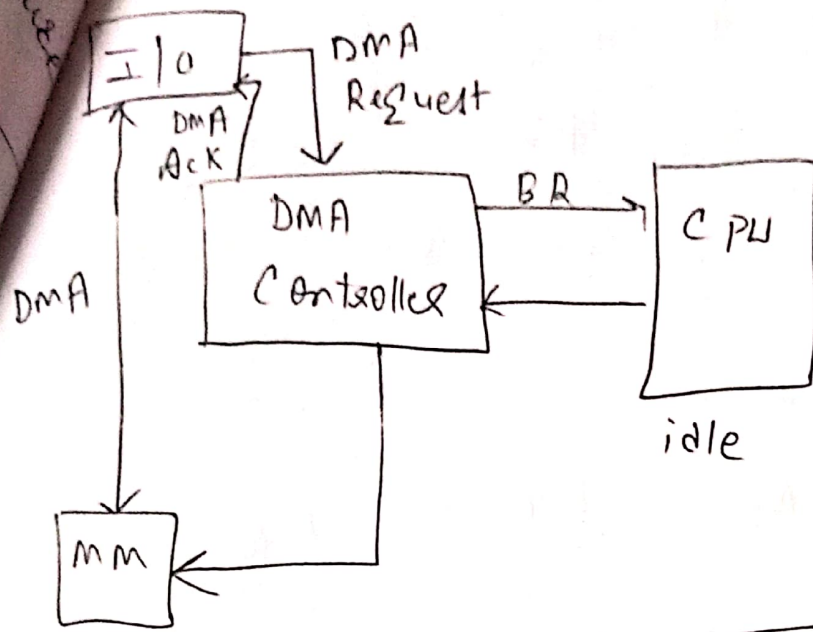
Transfer

① Burst or Block transfer
(For fast devices)

② Cycle stealing

Transfers one data word at a time, after which buses are returned to CPU.

CPU in idle state (DMA working)



UNIT - 3 (continued)

* The architecture of a computer system is the way hardware components are connected together to form a computer system whereas computer organization is concerned with the structure & behaviour of a computer system.

CA → Deals with high level design issues.
It involves instruction sets, addressing modes, data types, cache optimization etc.

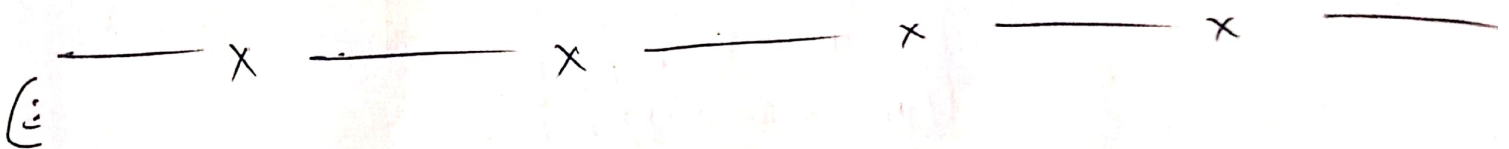
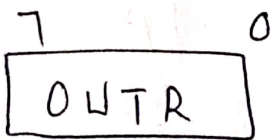
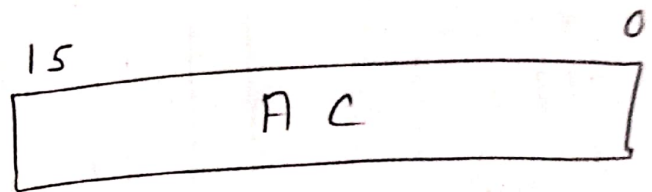
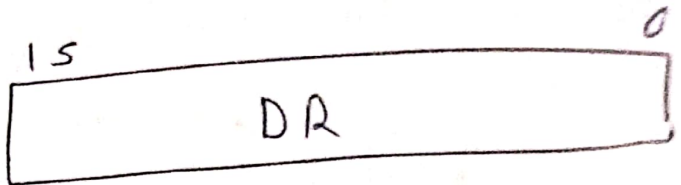
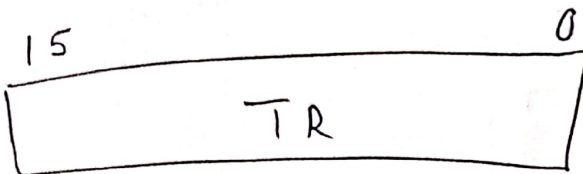
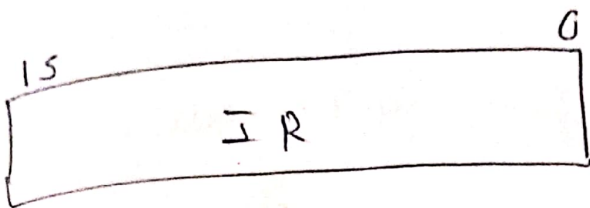
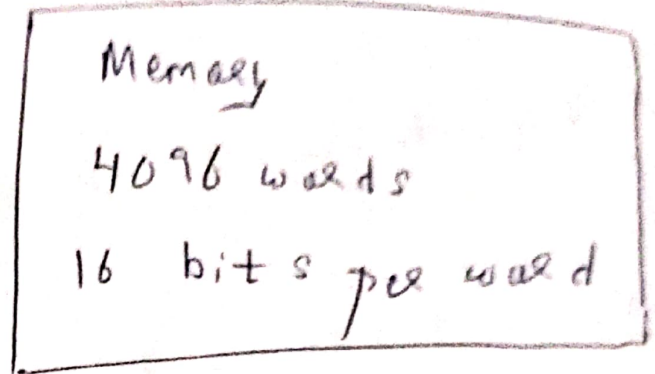
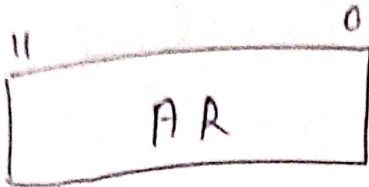
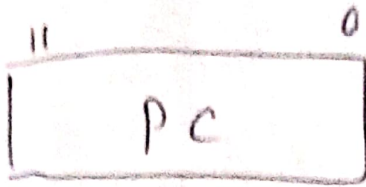
CO → Deals with low level design issues.
It involves physical components (ckt design, address, signals, peripherals).

Computer Registers

Type of computer memory to accept, store and transfer data & instructions that are being immediately by CPU. CPU registers are called processor registers.

Register	Symbol	No. of bits	Function
Data Register	DR	16	Holds memory operand.
Address Register	AR	12	Holds address for the memory.
Accumulator	AC	16	Processor register
Instruction Register	IR	16	Holds instruction code.
Program Counter	PC	12	Holds address of the instruction.
Temporary Register	TR	16	Holds temporary data
I/P Register	INPR	8	Carries I/P character
O/P Register	OUTR	8	Carries o/p character

Register & Memory configuration of a basic com)



General computer architecture

store program control concept

Flynn's classification

Non-Neuman model

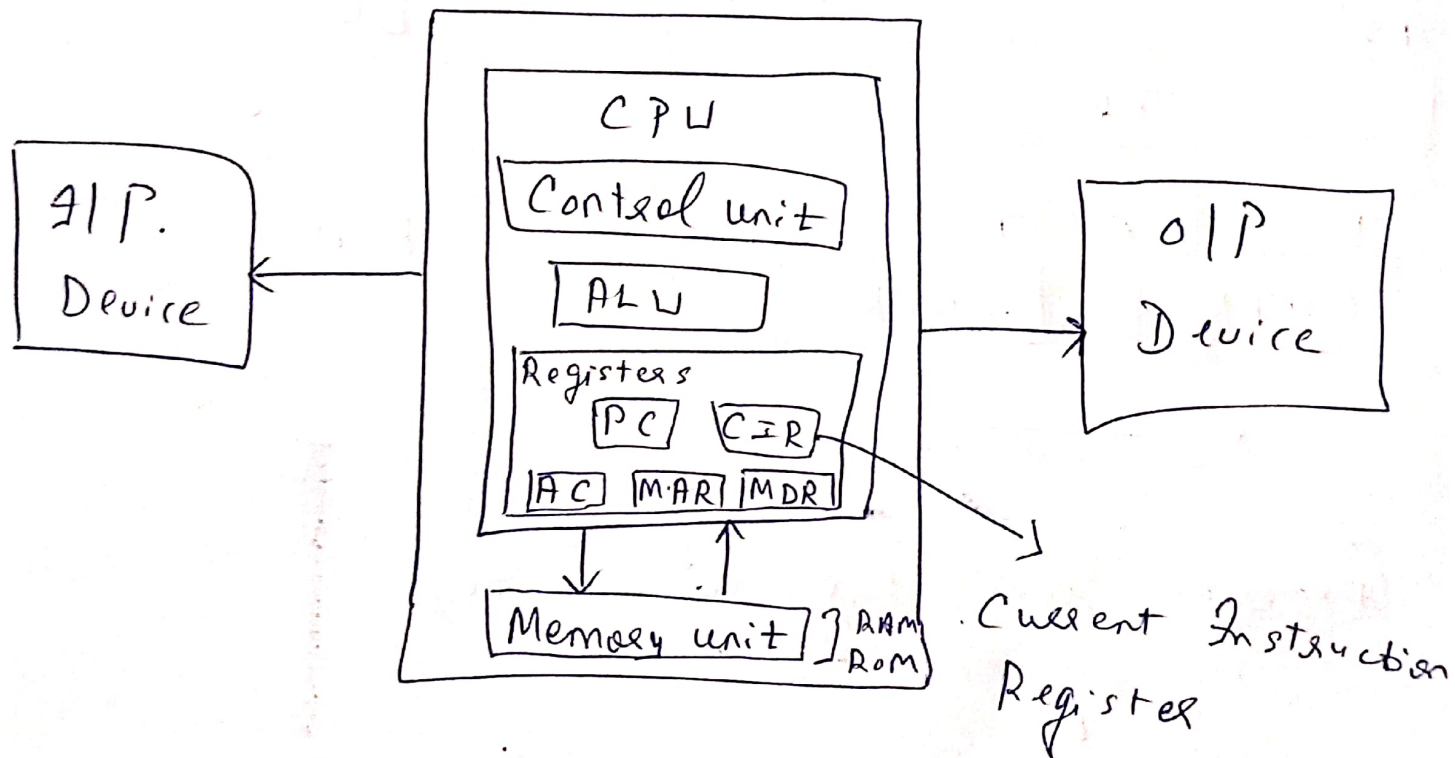
General purpose system

Parallel processing

2 SP concept refers to storage of instructions in computer memory to enable it to perform a variety of tasks in sequence or intermittently.
(irregular intervals)

① Van Newman based computer

* Here instructions, data & program data are stored in the same memory.

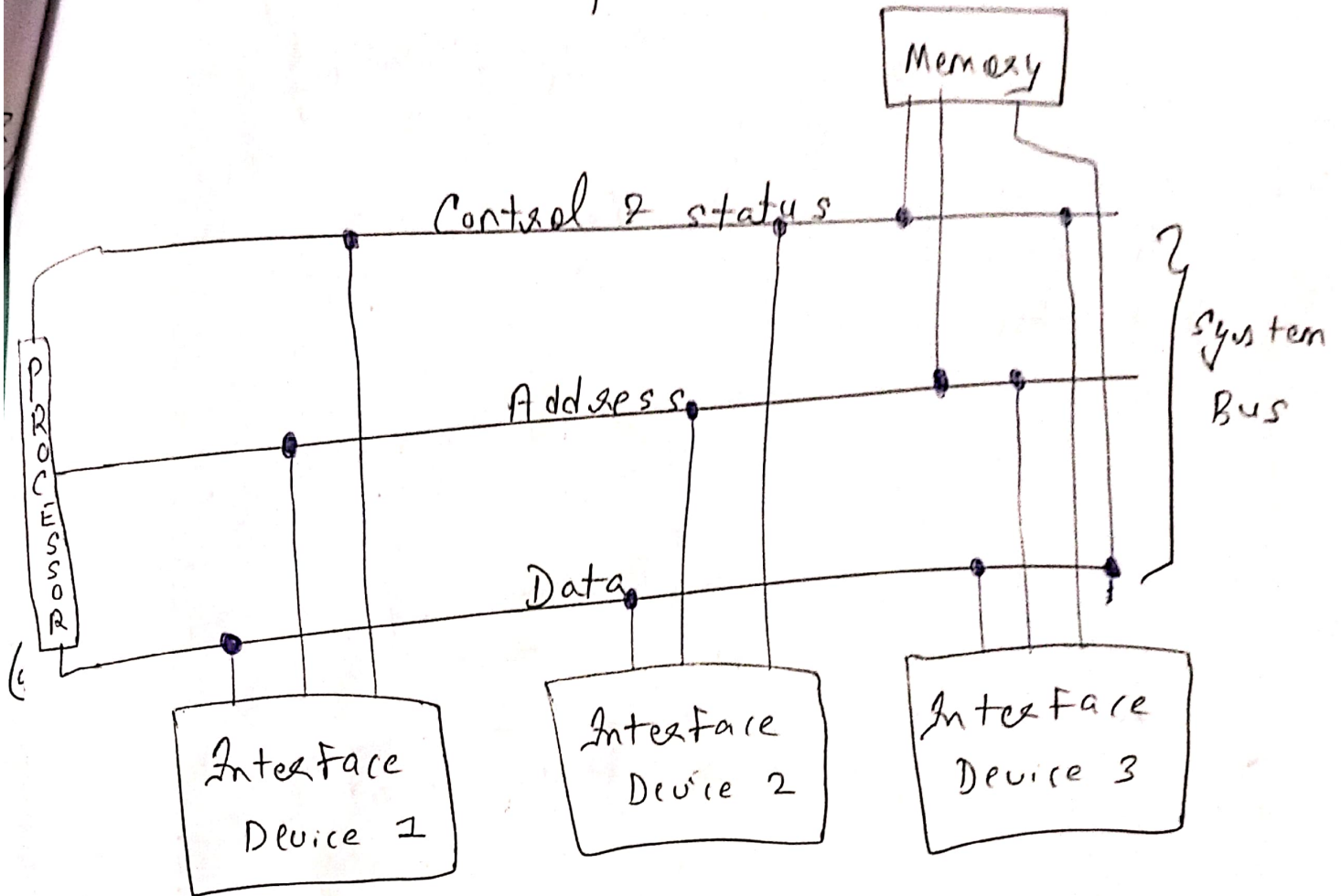


* Buses :- Means by which info is shared b/w the registers in a multiple register system.
Data / Address / control

② General Purpose system

Modified version of Von - Newman architecture.

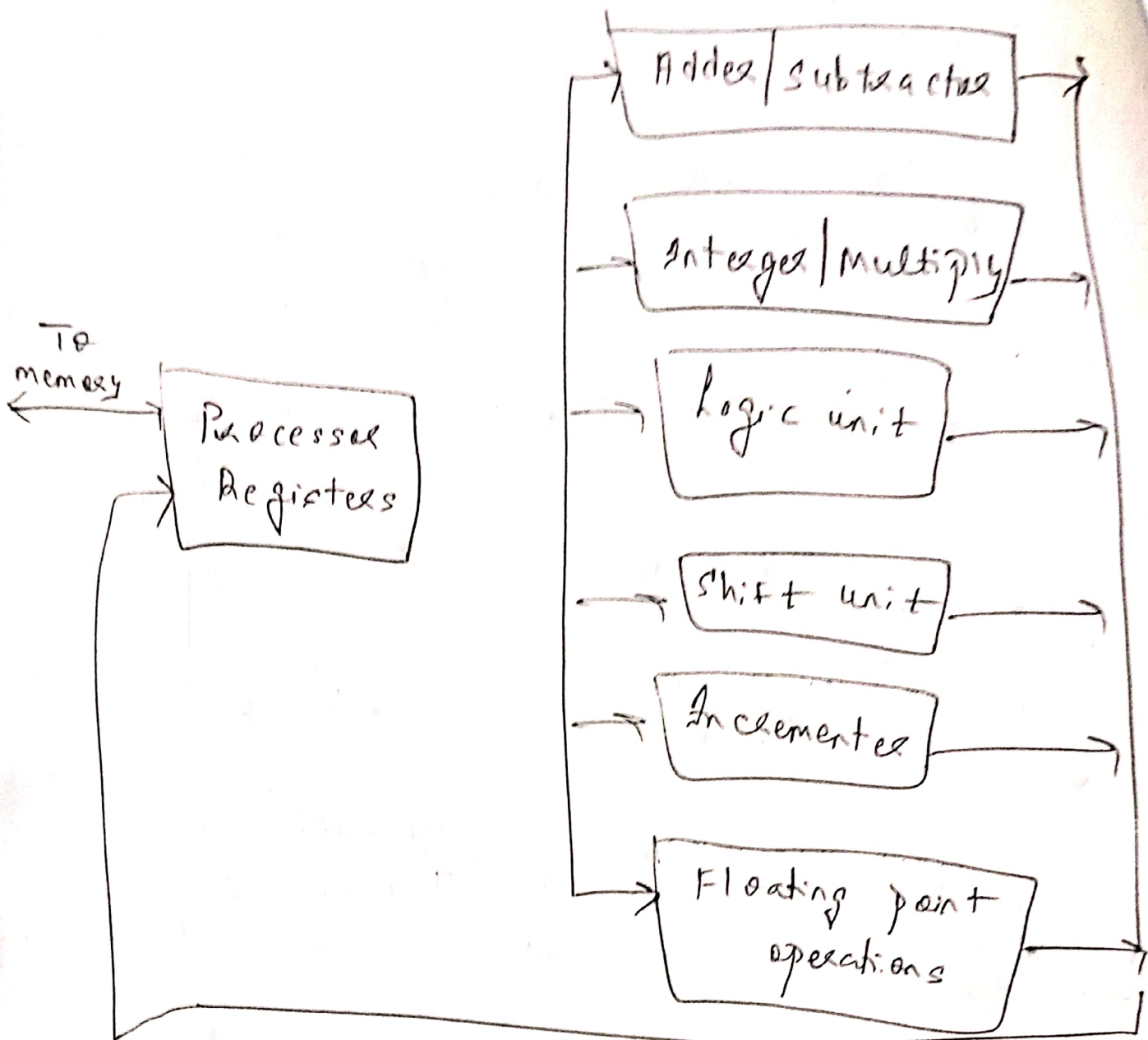
A general purpose computer system is a modern day architectural representation.



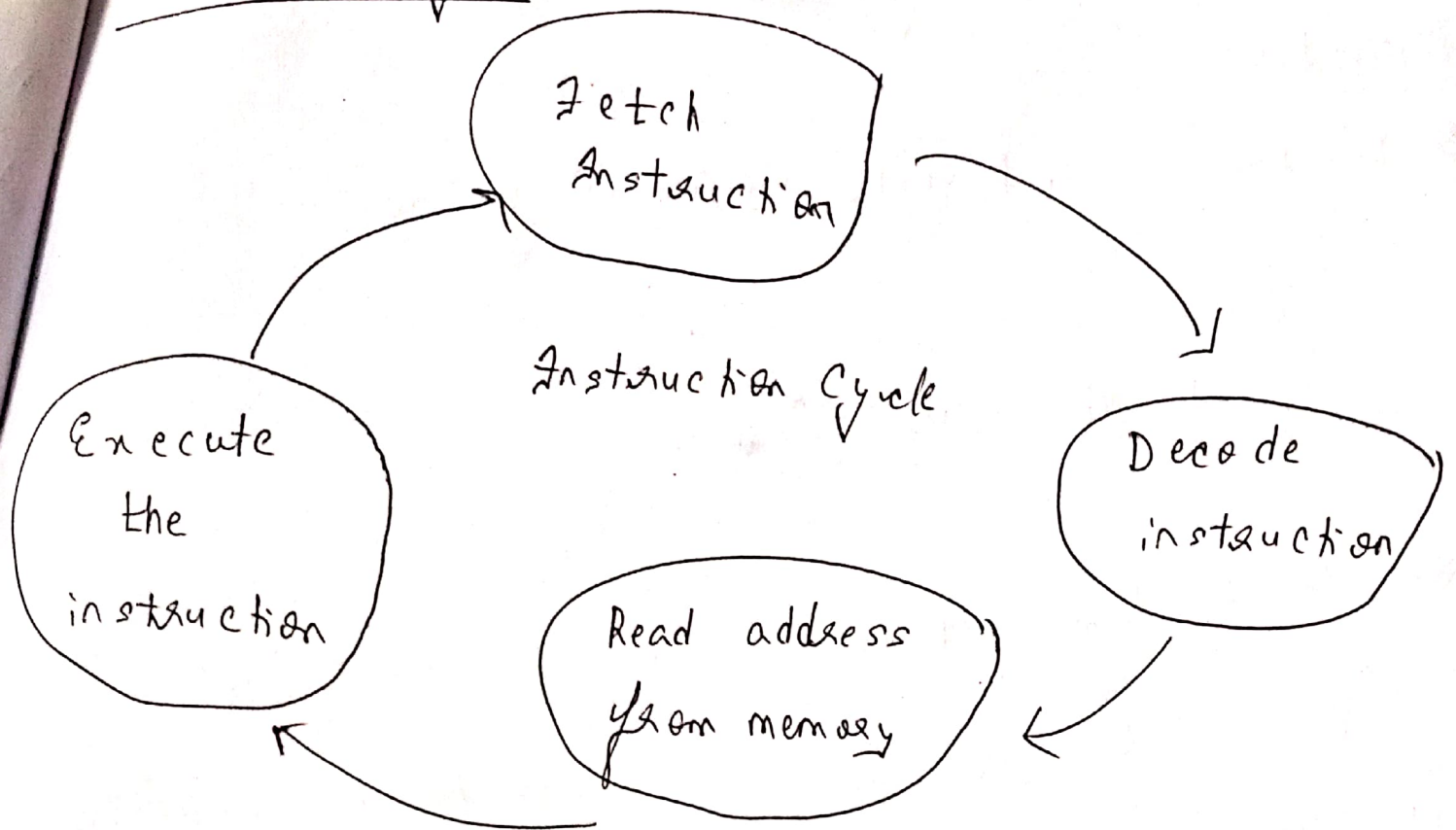
③ Parallel Processing :-

→ Achieves simultaneous data processing & thus faster execution time.

④



Instruction Cycle

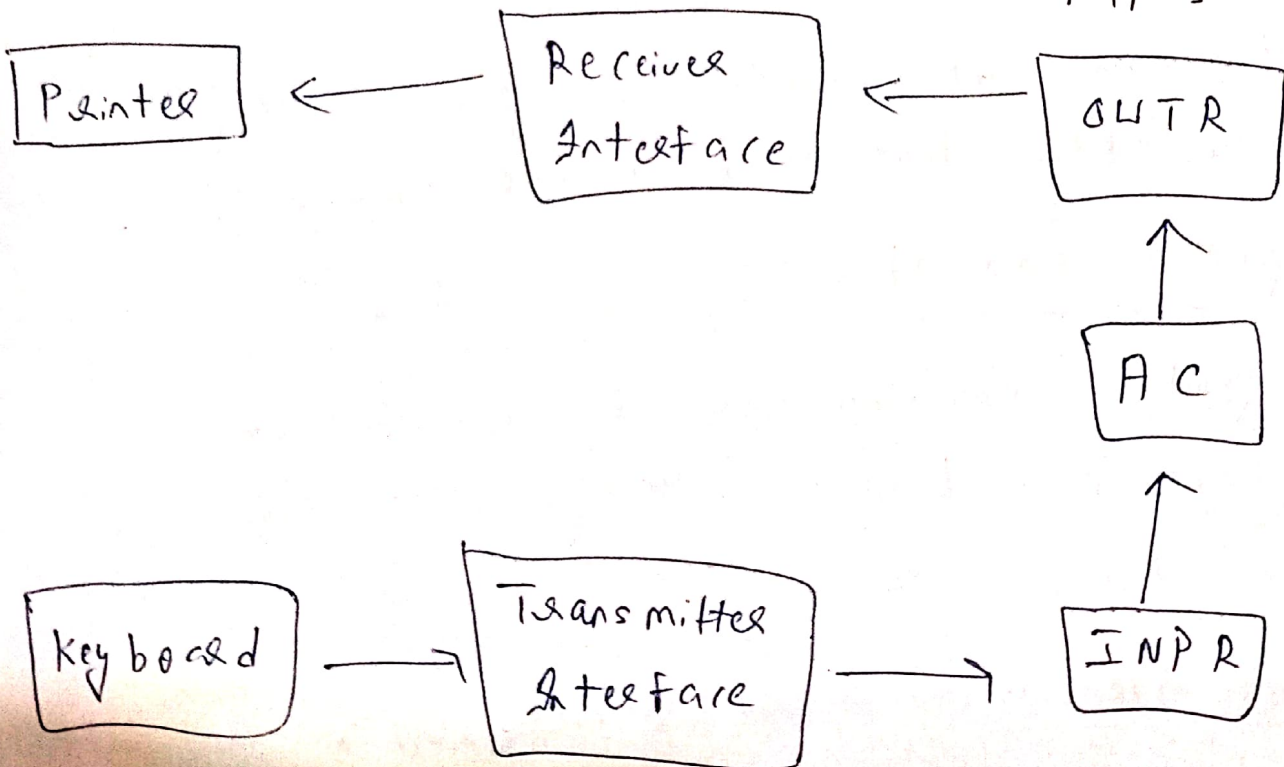


I/P O/P Configuration for a basic comp^r

I/P O/P Terminal

Serial commⁿ
interface

Computer Registers
&
F/F's



Computer

Instruction → Memory reference instanc

↓
ALU OP instanc

Instruction set completeness

A set of instructions are said to be complete if the computer includes the following: - (some of them)

- (1) Arithmetic, logical & shift instructions.
- (2) Instructions for moving infoⁿ to & from memory & processor registers
- (3) ALU & OP instructions
- (4) Instructions which control the program together with instructions that check status conditions.

Register Transfer Language (RTL)

The symbolic notation used to describe the micro-operation transfers amongst registers is called RTL.

micro operation: - Operations performed on the data stored in registers.

- * RTL is symbolic representation of notations used to specify the sequence of micro-operations.
- Register transfer means the availability of new logic ckt's that can perform a stated micro-operation & transfer the result of the operation to the same or another register.
- * Word language is borrowed from programmer's.
- * Programming language is a procedure for writing symbols.

②

Features of RTL

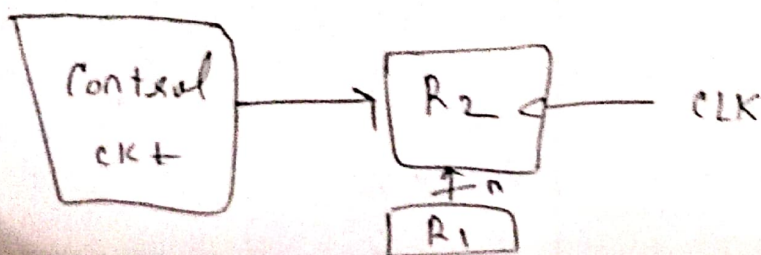
- ① Symbolic language
- ② Tool for describing internal organization of digital computer.
- ③ It is used to facilitate the design process.

* If $(P=1)$ then $(R_2 \leftarrow R_1)$; Here P is a control

④ R_1 signal.

or

$P: R_2 \leftarrow R_1$



Bus & Memory Transfers

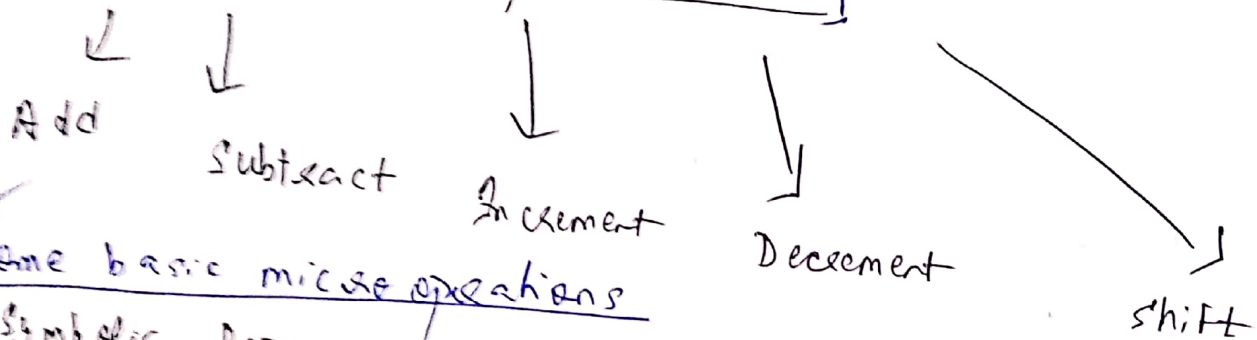
* Read: $DR \leftarrow M[AR]$

() Transfer of infoⁿ from memory unit to user end.

* Write: $M[AR] \leftarrow R1$

() It causes a transfer of infoⁿ from register R1 into the memory word (M) selected by address register (AR).

Arithmetic Micro operations



* Some basic micro operations

Symbolic Representation

Description

$$R3 \leftarrow R1 + R2$$

Add

$$R3 \leftarrow R1 - R2$$

Sub

$$R2 \leftarrow R2'$$

1's comp

$$R2 \leftarrow R2' + 1$$

2's comp

$$R3 \leftarrow R + R2' + 1$$

$$R1 \leftarrow R1 + 1$$

Increment

$$R1 \leftarrow R1 - 1$$

Decrement

* mul & Div are valid operations but they are not basic micro-operations.

↓
Repeated addition

↓
Repeated subtraction

Logic microoperations

AND gate $C \leftarrow A \cap B$

OR gate $C \leftarrow A \cup B$

Shift Microoperations

$A \leftarrow \text{shl } B$

$C \leftarrow \text{shr } A$

(— x — x — x — x — x —)

Design of central unit

} Do the Block diagrams

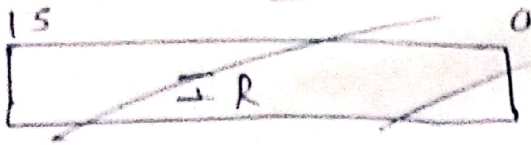
↓
Hardwired control

↓
Microprogrammed control

↓
Involves control logic to be implemented with gates, FIF's, decoders etc.

↓
Microoperations are performed by executing a program.

Hardwired control unit



Hardwired control unit

- ① Induces the control signals required for the processor.
- ② It is quicker.
- ③ Hard to modify.
- ④ More expensive as compared to MCU.
- ⑤ It faces difficulty in managing the complex instructions because ckt design is also complex.
- ⑥ It can use limited instructions.

Microprogrammed CU

Induces control signals through micro instructions.

Slower than HCU.

Easy to modify.

Affordable as compared to HCU.

Easily manages complex instructions.

It can generate control signals for many instructions.

Horizontal & Vertical Microprogramming

Horizontal signals are represented in a decoded binary format i.e. n control signals require n bit encoding.

Vertical signals are represented in the encoded binary format. n control signals require $\log_2 n$ bit encoding.

Horizontal microprog CW

(1) supports longer control word.

(2) High degree of parallelism.

(3) No additional HW required.

(4) Faster.

(5) More flexible.

(6) Uses horizontal microinstruction.

(7)

Vertical microprog CW

shorter CW.

Low degree.

Additional HW required in form of decoders to generate control signals.

Slower.

Less flexible.

Vertical microinstruction.

Instruction sets :- Group of commands for a CPU in machine language.

Example of some instructions

ADD :- Add two no's.

JUMP :- Jump to designated RAM address.

LOAD :- Load info from RAM to CPU.

Types of instruction set

↓
RISC



Computers which use fewer instructions with simple constructs so that they can be executed much faster within the CPU without having to use memory as often.

- * Few instructions & addressing modes.

- * single cycle instruction execution.

- * Hardwired rather than up.

↓
CISC



CISC is a computer where a single instruction can perform numerous low level operations.

- * Large number of instructions.

- * Large variety of addressing modes.

- * Variable length instruction formats.

- * Goal of CISC is to provide a single machine instruction for each statement that is written in high level language.

Instruction format :-

Instruction format type	Instruction Format	Example					
Zero address instruction	<table><tr><td>Mode</td><td>opcode</td></tr></table>	Mode	opcode	CMA CMF (Complement F) (overflow bit)			
Mode	opcode						
One address instruction	<table><tr><td>Mode</td><td>opcode</td><td>operand</td></tr></table>	Mode	opcode	operand	ADD 06H LDA 20H		
Mode	opcode	operand					
Two " "	<table><tr><td>Mode</td><td>opcode</td><td>operand 1</td><td>operand 2</td></tr></table>	Mode	opcode	operand 1	operand 2	MOV R1, R2 ADD AX, BX	
Mode	opcode	operand 1	operand 2				
Three " "	<table><tr><td>Mode</td><td>opcode</td><td>operand 1</td><td>operand 2</td><td>operand 3</td></tr></table>	Mode	opcode	operand 1	operand 2	operand 3	ADD R1, R2, R3 SUB R1, R2, R3
Mode	opcode	operand 1	operand 2	operand 3			

* Memory reference instruc	→	<table border="1"><tr><td>mode</td><td>opcode</td><td>Mem address</td></tr></table>	mode	opcode	Mem address
mode	opcode	Mem address			
* Register " "	→	<table border="1"><tr><td>mode</td><td>opcode</td><td>Register</td></tr></table>	mode	opcode	Register
mode	opcode	Register			
* I/O " "	→	<table border="1"><tr><td>mode</td><td>opcode</td><td>I/O operation</td></tr></table>	mode	opcode	I/O operation
mode	opcode	I/O operation			

* Interrupts

* Addressing modes ^{→ w.r.t (8085 µP)} :- The way of specifying data to be operated by an instruction.

Types

- ① Immediate :-
MOV B, 45
LXI H, 3050
JMP address
- ② Register addressing :-
MOV A, B
ADD B
INR A
- ③ Direct :-
LDA 2050
LHLD address
IN 35 (read data from port whose address is 35)
- ④ Register Indirect :-
MOV A, M
LDAX B (move BC reg contents to acc)
STAX B
} store acc contents to B-C register.

(5) Implied/Implicit Addressing ↓

CMA

RRC

RLC

operand is hidden & data
to be operated is
available in instruction
itself.